BÖLÜM 2 / MODULE 2

Yük Dengeleme, Ağ Protokolleri ve API Ağ Geçitleri

Load Balancing, Networking Protocols & API Gateways

Trafiği akıllıca yönlendirme sanatı: Anycast DNS, L4 vs L7 Load Balancer, gRPC vs WebSockets, API Gateway deseni ve Dağıtık Rate Limiting algoritmaları.

The art of traffic routing: Anycast DNS, Layer 4 vs Layer 7 load balancing, gRPC vs WebSockets, API Gateway patterns, and Distributed Rate Limiting algorithms.

7

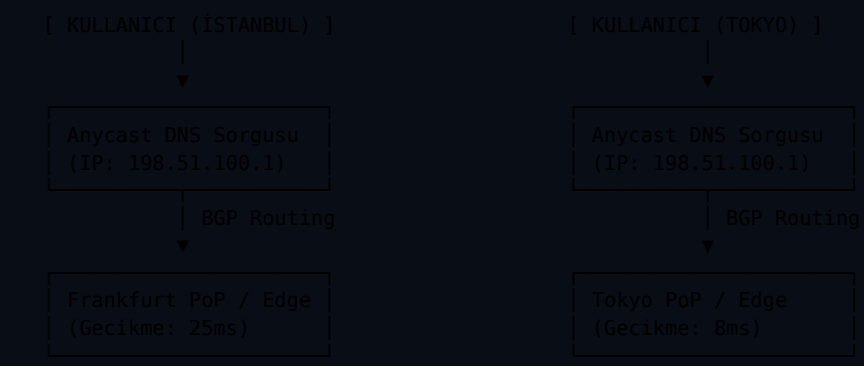
BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

DNS ve Coğrafi Yönlendirme: Anycast, Geo-DNS ve Latency-Based Routing

DNS & Geo-Routing: Anycast, GeoDNS & Latency Routing

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Dünyanın neresinde olursanız olun acil ambulans çağırmak için tek bir numara (örneğin 112) çevirirsiniz. Ama İstanbul'dan aradığınızda telefonunuz Tokyo'daki hastaneye değil, sokağınızın başındaki istasyona bağlanır! İşte `Anycast DNS` tam olarak budur: Bütün dünyaya tek bir IP adresi ilan edersiniz (BGP yönlendirme ile); internet yönlendiricileri kullanıcının isteğini ona ışık hızıyla en yakın olan veri merkezine teslim eder!

🇬🇧 ENGLISH (ELI5)

No matter what neighborhood you live in, dialing an emergency number routes you to the closest ambulance garage, not one across the Pacific Ocean! That is `Anycast BGP Routing`. Multiple servers across the globe announce the exact same IP address. Autonomous internet routers automatically steer a user's packets along the shortest physical BGP path to their nearest edge Point of Presence (PoP)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Geleneksel Unicast mimarisinde her sunucu tekil bir IP adresine sahiptir. Anycast'te ise dünyanın dört bir yanındaki onlarca veri merkezi BGP (Border Gateway Protocol) üzerinden aynı IP bloğunu anons eder. İstemci DNS sorgusu attığında en yakın PoP (Point of Presence) yanıt verir. Geo-DNS ise istemcinin IP adresine bakarak coğrafi konuma özel farklı IP'ler döner (örneğin ABD kullanıcısına us-east, Avrupa kullanıcısına eu-west IP'si). Latency-Based Routing ise AWS Route 53 gibi sistemlerin küresel ağ gecikmelerini sürekli ölçerek en düşük RTT sunan bölgeye yönlendirmesidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Unicast binds an IP address uniquely to one machine. In Anycast, distributed PoPs globally announce identical IP prefixes via BGP. Routers forward packets toward the topologically nearest network hop. GeoDNS inspects client recursive resolver IPs to synthesize geographically tailored responses. Latency-Based Routing (e.g. AWS Route 53) continuously probes edge-to-region RTTs to resolve DNS to the lowest-latency origin datacenter.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Simüle Edilmiş Coğrafi DNS Çözümleyici (GeoDNS Resolver)
import java.util.Map;

public record GeoDnsResolver(Map<String, String> regionDatacenters, String defaultDatacenter) {
    public String resolveTargetIp(String clientCountryCode) {
        return regionDatacenters.getOrDefault(clientCountryCode, defaultDatacenter);
    }
}

void main() {
    var geoDns = new GeoDnsResolver(Map.of(
        "TR", "195.175.25.10 (Frankfurt Edge)",
        "JP", "210.140.10.20 (Tokyo Edge)",
        "US", "198.51.100.5 (Virginia Edge)"
    ), "198.51.100.5 (Default US-East)");

    IO.println("Türkiye'den gelen istek -> " + geoDns.resolveTargetIp("TR"));
    IO.println("Japonya'dan gelen istek -> " + geoDns.resolveTargetIp("JP"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

DNS tabanlı yönlendirmede DNS önbellekleme (TTL - Time To Live) riskini unutmayın! TTL 300 saniye (5 dakika) ise, bir veri merkezi çöktüğünde kullanıcıların DNS önbelleği 5 dakika boyunca ölü sunucuya gitmeye devam eder. Bu yüzden DNS tek başına anlık failover için yetersizdir; önüne Anycast IP konmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

DNS TTL caching delays failover! If TTL is 300 seconds, clients continue sending traffic to a crashed datacenter for up to 5 minutes. Never rely solely on DNS for sub-second failover; front it with Anycast BGP routing.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Anycast, DDoS saldırılarını küresel PoP noktalarına dağıtarak soğurur.

• Geo-DNS ve Latency-Based Routing kullanıcıyı fiziksel olarak en yakın merkeze bağlar.

• Düşük TTL failover süresini kısaltır ancak kök DNS sunucularına binen sorgu yükünü artırır.

🎯 Key Takeaways for the Interview

• Anycast absorbs multi-terabit DDoS traffic by dispersing packets across dozens of edge PoPs.

• GeoDNS and latency routing minimize round-trip times by binding users to adjacent regions.

• Lowering DNS TTL accelerates disaster failover at the expense of higher DNS query load.

8

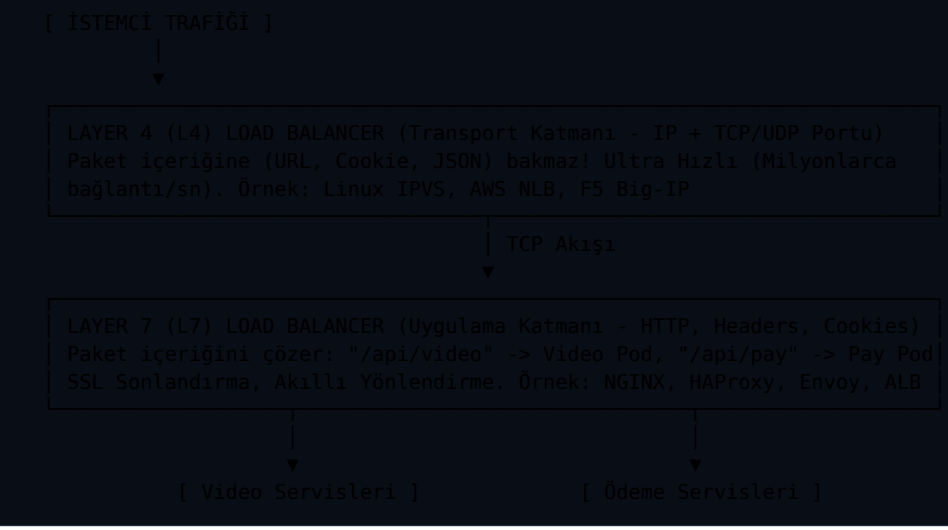
BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

Yük Dengeleyiciler (L4 vs L7): TCP vs HTTP, NGINX, HAProxy, AWS ALB

Load Balancers (L4 vs L7): TCP vs HTTP, NGINX, HAProxy, AWS ALB

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

L4 ve L7 yük dengeleyiciyi bir postanenin iki farklı görevlisi gibi hayal edin: L4 görevlisi, sadece zarfın üzerindeki sokak numarasına bakar (IP ve Port). Zarfın içini ASLA açmaz; paketi gözü kapalı ışık hızında doğru binaya fırlatır! L7 görevlisi ise zarfı açar, içindeki mektubu okur (HTTP URL'i, Cookie, Başlıklar). 'Aaa bu bir fatura ödemesiymiş, bunu muhasebe masasına götüreyim; bu bir video talebiymiş, bunu sinema salonuna göndereyim' der. L4 süper hızlıdır, L7 ise süper zekidir!

🇬🇧 ENGLISH (ELI5)

Think of L4 and L7 load balancers as two mail sorting clerks: The L4 clerk only glances at the building address on the outside envelope (IP and TCP Port). They NEVER open the envelope; they fling packets at blistering wire speed! The L7 clerk tears the envelope open, reads the letter (HTTP Headers, Cookies, URL paths), and decides: 'This letter says /video, send it to the media cluster; this says /checkout, route to the payment gateway.' L4 offers raw throughput; L7 delivers smart content routing!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

OSI 4. Katman (L4), Taşıma katmanında (TCP/UDP) çalışır. NAT (Network Address Translation) veya DSR (Direct Server Return) kullanır. HTTP paketini çözmediği (SSL terminasyonu yapmadığı) için CPU maliyeti çok düşüktür; saniyede milyonlarca eşzamanlı TCP bağlantısını yönetebilir. OSI 7. Katman (L7), Uygulama katmanında çalışır. TLS/SSL el sıkışmasını sonlandırır (SSL Termination), HTTP istek başlıklarını, URI yolunu (`/users` vs `/orders`), JWT tokenlarını ve Cookie oturumlarını inceler. Bu sayede mikroservis yönlendirmesi, A/B testi ve rate limiting yapabilir; ancak CPU ve bellek maliyeti daha yüksektir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Layer 4 (L4) operates on TCP/UDP transport segments using NAT or Direct Server Return (DSR). Because it avoids terminating TLS or parsing HTTP buffers, L4 introduces microsecond latency and scales to tens of millions of concurrent connections (AWS NLB, Maglev). Layer 7 (L7) operates on application-level HTTP/HTTPS semantics. It terminates TLS, parses headers, inspects cookies, and routes requests based on URI paths (`/stream` vs `/auth`). L7 enables canary routing, sticky sessions, and rate limiting at higher CPU overhead.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: L7 İçerik Tabanlı Yönlendirme Simülasyonu (Path-Based Routing)
public class Layer7Router {
    public record HttpRequest(String path, String method, String clientIp) {}

    public String routeRequest(HttpRequest req) {
        // Java 25 Pattern Matching ile L7 URL Yönlendirmesi:
        return switch (req.path()) {
            case String p when p.startsWith("/api/v1/auth")     -> "AUTH_SERVICE_CLUSTER";
            case String p when p.startsWith("/api/v1/payments") -> "PAYMENT_PCI_CLUSTER";
            case String p when p.startsWith("/static/")         -> "CDN_S3_STORAGE";
            default                                             -> "CORE_BACKEND_CLUSTER";
        };
    }
}

void main() {
    var router = new Layer7Router();
    var payReq = new Layer7Router.HttpRequest("/api/v1/payments/charge", "POST", "10.0.0.1");
    IO.println("İstek Hedefi: " + router.routeRequest(payReq));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Tüm trafiği doğrudan tek bir NGINX L7 load balancer'a bağlarım' demeyin! Yüksek ölçekte (milyonlarca QPS) mimari standart şöyledir: Anycast DNS -> L4 Load Balancer (AWS NLB / Maglev) -> L7 Load Balancer Filosu (NGINX / Envoy) -> Mikroservisler.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never funnel millions of raw internet connections directly into a single L7 NGINX node. The enterprise standard is a two-tier hierarchy: Anycast DNS -> L4 NLB cluster -> Fleet of autoscaled L7 Envoy/NGINX proxies.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• L4 paket içeriğine bakmaz, ultra yüksek verim (throughput) ve düşük gecikme sunar.

• L7 HTTP başlıklarına ve URL'e göre akıllı mikroservis yönlendirmesi yapar.

• Modern mimarilerde L4 önde donanım kalkanı, L7 arkada uygulama beyni olarak birlikte kullanılır.

🎯 Key Takeaways for the Interview

• L4 routes on IP:Port tuples, offering extreme connection throughput and sub-millisecond latencies.

• L7 inspects HTTP/gRPC semantics for intelligent path-based and header-based routing.

• Two-tier load balancing places L4 at the edge fronting a dynamic fleet of L7 proxies.

9

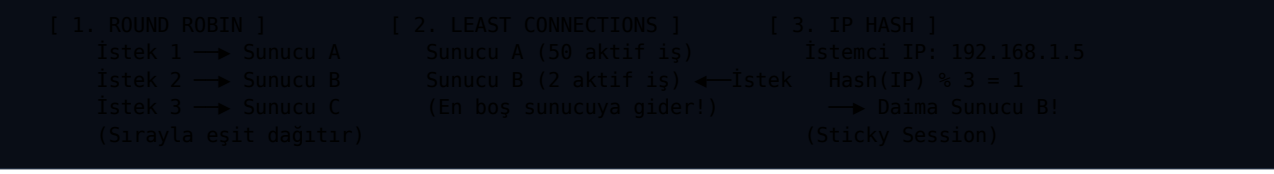
BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

Yük Dengeleme Algoritmaları: Round Robin, Least Connections, Weighted ve IP Hash

Load Balancing Algorithms: Round Robin, Least Conn, Weighted & IP Hash

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir bankada 3 gişe memuru olduğunu düşünün. Round Robin, sıra numarasını sırayla 1-2-3-1-2-3 diye memurlara dağıtan robot gibidir; ancak 1. memurun önündeki müşteri 1 saat sürecek devasa bir miras işlemi yapıyorsa o gişenin önü kilitlenir! Least Connections ise akıllı bir şeftir: Kafasını kaldırır, 'Hanginizin önünde daha az bekleyen insan var?' diye bakar ve yeni müşteriyi en boş memura gönderir. IP Hash ise 'Ali Bey sen daima 2 numaralı memura git' diyen kuraldır (aynı kullanıcı hep aynı sunucuya gider)!

🇬🇧 ENGLISH (ELI5)

Picture 3 bank tellers. Round Robin is a naive ticket dispenser handing turns sequentially: 1, 2, 3, 1, 2, 3. But if Teller 1 is stuck in a 2-hour corporate audit, their desk clogs completely while Teller 2 sits bored! Least Connections is a vigilant floor manager who checks: 'Who currently has the fewest active clients?' and dispatches the newcomer there. IP Hash calculates a hash of the customer's ID, consistently assigning that specific person to the exact same teller every visit (Session Stickiness)!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Yük dengeleme algoritmaları ikiye ayrılır: 1) Statik Algoritmalar: Sunucu yükünü umursamaz. Round Robin (sırayla), Weighted Round Robin (güçlü sunucuya daha fazla ağırlık, ör. 8 core makineye 4x istek), IP Hash (istemci IP'sine göre yapışkan oturum - Sticky Session). 2) Dinamik Algoritmalar: Gerçek zamanlı sunucu metriğini izler. Least Connections (aktif TCP bağlantısı en az olan sunucuya gönderir - uzun süren işlemler ve WebSockets için idealdir), Weighted Least Connections, Least Response Time (en hızlı yanıt veren sunucuyu tercih eder).

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Balancing algorithms categorize into: 1) Static Schemes: Agnostic to real-time server load. Round Robin (simple cycle), Weighted Round Robin (proportional weights assigned to heftier bare-metal nodes), IP/Consistent Hashing (binds client IP to persistent backend instances for stateful caching). 2) Dynamic Schemes: Real-time telemetry feedback. Least Connections (routes to node with minimum active sockets —vital for persistent WebSockets and heavy database queries), Weighted Least Connections, and Least Response Time (prioritizes instances demonstrating lowest TTFB).

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Thread-Safe Weighted Round Robin & Least Connections Simülatörü
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class LoadBalancingSim {
    public record BackendNode(String id, int weight, AtomicInteger activeConnections) {}

    // 1. Round Robin
    public static class RoundRobinBalancer {
        private final List<BackendNode> nodes;
        private final AtomicInteger counter = new AtomicInteger(0);

        public RoundRobinBalancer(List<BackendNode> nodes) { this.nodes = nodes; }

        public BackendNode getNextNode() {
            int idx = Math.abs(counter.getAndIncrement() % nodes.size());
            return nodes.get(idx);
        }
    }

    // 2. Least Connections
    public static BackendNode selectLeastConnection(List<BackendNode> nodes) {
        return nodes.stream()
                .min((a, b) -> Integer.compare(a.activeConnections().get(),
b.activeConnections().get()))
                .orElseThrow();
    }
}

void main() {
    var n1 = new LoadBalancingSim.BackendNode("Node-A", 1, new AtomicInteger(42));
    var n2 = new LoadBalancingSim.BackendNode("Node-B", 1, new AtomicInteger(5));
    var n3 = new LoadBalancingSim.BackendNode("Node-C", 1, new AtomicInteger(18));

    var best = LoadBalancingSim.selectLeastConnection(List.of(n1, n2, n3));
    IO.println("Least Connection Seçimi: " + best.id() + " (Aktif: " + best.activeConnections() +
")");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

IP Hash ile oturum sabitleme (Sticky Session) dağıtık mimarilerde bir anti-pattern'dir! Bir sunucu çöktüğünde o sunucuya bağlı tüm kullanıcıların oturumu uçar ve yük dengesizleşir. Modern MAANG mimarilerinde sunucular tamamen durumsuz (Stateless) tutulur, oturumlar Redis'te saklanır ve Least Connections kullanılır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Sticky Sessions via IP Hash represent an architectural anti-pattern in cloud microservices. If an instance restarts, connected users lose state. Keep backends strictly stateless, store tokens in Redis, and leverage Least Connections.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Kısa süren tipik REST istekleri için Weighted Round Robin idealdir.

• Uzun süren veritabanı veya WebSocket bağlantıları için Least Connections şarttır.

• Durumsuz (stateless) tasarım sayesinde sunucu çökmeleri istemciyi etkilemez.

🎯 Key Takeaways for the Interview

• Weighted Round Robin excels for short-lived, uniform HTTP REST transactions.

• Least Connections is critical for long-lived WebSockets and compute-heavy transactions.

• Stateless servers combined with external token stores render sticky sessions obsolete.

10

BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

Ağ İletişim Protokolleri: HTTP/1.1 vs HTTP/2 vs HTTP/3 QUIC, WebSockets, gRPC, SSE

Network Protocols: HTTP/1.1 vs HTTP/2 vs HTTP/3, WebSockets, gRPC, SSE

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ PROTOKOL EVRİMİ ]
HTTP/1.1 : Tek TCP bağlantısında tek istek (Head-of-Line Blocking!)
HTTP/2   : Tek TCP üzerinde Multiplexing (Binary Framing, Header Compression)
HTTP/3   : UDP Tabanlı QUIC! (Sıfır TCP Head-of-Line, 0-RTT El Sıkışma!)

[ İKİ YÖNLÜ vs TEK YÖNLÜ İLETİŞİM ]
WebSockets : Tam İki Yönlü (Full-Duplex) Sürekli TCP Kanalı (Chat, Finans)
SSE (Event): Sunucudan İstemciye Tek Yönlü Canlı Veri Akışı (Borsa, Skor)
gRPC       : HTTP/2 Üzerinde Protobuf ile Ultra Hızlı Mikroservis İletişimi
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

HTTP/1.1 tek şeritli dar bir otoyola benzer; öndeki kamyon arıza yaparsa arkadaki 100 araba bekler (Head-of-Line Blocking). HTTP/2 bu yola 10 şerit ekler; aynı kablodan aynı anda onlarca veri akar. HTTP/3 (QUIC) ise asfaltı kaldırıp manyetik raylı trene geçmektir: TCP'nin hantal onay mekanizmalarını atar, UDP üzerinden sıfır gecikmeyle uçar! WebSockets iki kişinin telefonla kesintisiz konuşmasıdır; SSE ise televizyondaki canlı haber bültenini tek yönlü dinlemektir!

🇬🇧 ENGLISH (ELI5)

HTTP/1.1 is a single-lane mountain road: if a lumber truck stalls, all 50 cars behind it freeze (Head-of-Line Blocking). HTTP/2 widens it into a 10-lane superhighway, multiplexing dozens of concurrent streams over one TCP pipe. HTTP/3 (QUIC) throws away asphalt entirely: it swaps fragile TCP for lightning-fast UDP with zero-RTT handshakes! WebSockets is an open telephone call (full-duplex chat); Server-Sent Events (SSE) is tuning into an emergency FM radio station (unidirectional live streaming)!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) HTTP/1.1: Metin tabanlıdır; her istek için yeni TCP veya keep-alive gerektirir. 2) HTTP/2: Binary framing katmanı getirir. Tek TCP bağlantısında stream multiplexing ve HPACK başlık sıkıştırması yapar; ancak paket kaybında TCP seviyesinde HoL blocking yaşar. 3) HTTP/3 (QUIC): Google'ın geliştirdiği UDP tabanlı protokoldür. Her akış (stream) birbirinden tamamen bağımsızdır; bir paket kaybolsa bile diğer akışlar etkilenmez. 4) gRPC: Google'ın HTTP/2 üzerinde Protocol Buffers (ikili serileştirme) kullanan, JSON'dan 7-10 kat daha hızlı ve tip güvenli RPC protokolüdür; iç mikroservis haberleşmesinin standardıdır.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) HTTP/1.1: Text-oriented with head-of-line blocking. 2) HTTP/2: Binary framing layer enabling multiplexed bidirectional streams over a shared TCP connection with HPACK header compression; still suffers TCP-level HoL blocking on packet drops. 3) HTTP/3 (QUIC): Replaces TCP with UDP-based transport, eliminating multi-stream HOL blocking and slashing handshakes to 0-RTT connection resumptions across mobile cell towers. 4) gRPC: High-throughput RPC framework using HTTP/2 multiplexing and Protocol Buffers serialization; de facto industry standard for microservice-to-microservice internal transit.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```java
// Java 25: gRPC vs REST Veri Serileştirme Karşılaştırma Modeli
// Protocol Buffers Mantığı (Java 25 Record ile Temsil Edilmiş Binary Kompakt Yapı)
public record UserProfileProto(long userId, String username, String email, int karma) {
    // JSON Boyutu: ~95 Byte  |  Protobuf İkili Boyutu: ~28 Byte (3.4 kat daha küçük!)
    public byte[] toMockProtobufBytes() {
        // İkili serileştirme simülasyonu: Ağ trafiğini ve serileştirme CPU yükünü minimize eder
        return ("B:" + userId + ":" + karma).getBytes();
    }
}

void main() {
    var user = new UserProfileProto(1049281L, "aliturgut", "ali@example.com", 942);
    IO.println("Simüle Protobuf Boyutu: " + user.toMockProtobufBytes().length + " bytes");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'İç mikroservisler arasında da JSON REST kullanırım' demeyin. Dış dünyadan istemciler için JSON REST uygundur; ancak iç mikroservisler arasında yüksek verim, düşük CPU tüketimi ve tip güvenliği için gRPC (Protobuf) tercih edilmelidir.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Don't default to JSON REST across internal microservices. Use REST at the perimeter API Gateway, but standardize on gRPC over HTTP/2 internally for compact binary footprints and code-generated contracts.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar 🎯 Key Takeaways for the Interview

- HTTP/3 QUIC mobil cihazların baz istasyonu değişiminde bile bağlantıyı koparmaz.
- gRPC ikili Protocol Buffers formatı ile JSON'a kıyasla ağ ve CPU tasarrufu sağlar.
- Canlı iki yönlü sohbet için WebSockets, tek yönlü canlı akışlar için SSE tercih edilir.

- HTTP/3 QUIC connection IDs migrate seamlessly across Wi-Fi and cellular switches without dropping.
- gRPC cuts payload serialization overhead by up to 70% compared to verbose JSON strings.
- Use WebSockets for bidirectional duplex streams (chat) and SSE for server pushes (tickers).

11

BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

API Gateway Deseni: Kimlik Doğrulama, SSL Sonlandırma ve Protokol Dönüşümü

The API Gateway Pattern: Auth, SSL Termination & Protocol Translation

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir ülkeye girerken sınır kapısındaki devasa pasaport kontrol binasını düşünün. Ülkedeki 100 farklı şehre gitmek isteyen her turist önce o tek kapıdan geçer: Pasaport kontrol edilir (Kimlik doğrulama), vizesi var mı bakılır (Yetkilendirme), valizinde yasaklı madde var mı taranır (Rate limiting & WAF). Şehirlere geçtikten sonra kimse size her dükkanda tekrar tekrar pasaport sormaz! API Gateway işte o sınırdaki akıllı pasaport kapısıdır!

🇬🇧 ENGLISH (ELI5)

Picture a nation's central border checkpoint terminal. Every foreign traveler heading to any of 50 internal cities must pass through this single customs facility: passports are verified (Authentication), visas are inspected (Authorization), and luggage is x-rayed for contraband (WAF & Rate Limiting). Once inside the country, individual shops don't re-examine your passport! The API Gateway is that secure border checkpoint!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

API Gateway deseni, mikroservis mimarilerinde tek bir giriş noktası (Single Entry Point) sunar. Görevleri: 1) SSL/TLS Termination: Ağır şifre çözme işlemini üstlenir, iç ağdaki mikroservislerin CPU yükünü düşürür. 2) Kimlik Doğrulama & Yetkilendirme: Gelen JWT/OAuth2 tokenını doğrular, kullanıcı kimliğini başlığa (`X-User-Id`, `X-Roles`) koyarak iç servislere aktarır. 3) Protokol Dönüşümü: İstemciden gelen HTTP/JSON isteklerini içeride gRPC veya Protobuf formatına dönüştürür. 4) İstek Birleştirme (API Composition): Tek bir mobil ekran için 5 farklı servisten veriyi paralel çekip tek bir JSON cevabı döner.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The API Gateway serves as the centralized reverse-proxy perimeter for microservices. Core capabilities: 1) SSL/TLS Termination: Offloads cryptographic handshakes from upstream worker pods. 2) Authentication & Token Introspection: Validates JWT/OAuth2 signatures at the edge, injecting sanitized user identity headers (`X-User-Id`, `X-Tenant-Id`) downstream. 3) Protocol Translation: Ingests public HTTP/REST and translates into internal high-speed binary gRPC calls. 4) Request Aggregation (BFF - Backend for Frontend): Batches parallel calls across disparate domains to prevent mobile network chattiness.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: API Gateway Filtre Zinciri (Pipeline Filter Pattern)
import java.util.function.Function;

public class ApiGatewayPipeline {
    public record GatewayRequest(String path, String token, String clientIp) {}
    public record GatewayContext(GatewayRequest request, String resolvedUserId, boolean authenticated)
{}

    public static Function<GatewayContext, GatewayContext> authFilter() {
        return ctx -> {
            if ("secret-jwt-token".equals(ctx.request().token())) {
                return new GatewayContext(ctx.request(), "usr_9981", true);
            }
            return new GatewayContext(ctx.request(), null, false);
        };
    }

    public static Function<GatewayContext, String> router() {
        return ctx -> ctx.authenticated()
            ? "FORWARD -> /internal" + ctx.request().path() + " [UID:" + ctx.resolvedUserId() + "]"
            : "HTTP 401 UNAUTHORIZED";
    }
}

void main() {
    var filter = ApiGatewayPipeline.authFilter().andThen(ApiGatewayPipeline.router());
    var validReq = new ApiGatewayPipeline.GatewayRequest("/orders", "secret-jwt-token", "1.2.3.4");
    IO.println(filter.apply(new ApiGatewayPipeline.GatewayContext(validReq, null, false)));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

API Gateway'in içine iş mantığı (Business Logic) YAZMAYIN! Gateway bir proxy'dir; fatura hesaplama veya veritabanı sorgusu gibi işler eklerseniz Gateway monolitik bir darboğaza ve tek hata noktasına (SPOF) dönüşür.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never inject core business logic into the API Gateway layer! A gateway must remain a lean transport proxy; bloating it with domain business logic creates a brittle distributed monolith bottleneck.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- API Gateway mikroservisleri dış dünyanın karmaşıklığından ve güvenlik risklerinden izole eder.
- SSL sonlandırma iç mikroservislerin CPU kapasitesini iş mantığına ayırmasını sağlar.
- Backend-for-Frontend (BFF) deseni mobil ve web istemcilerine özel optimize edilmiş gateway'ler kurar.

🎯 Key Takeaways for the Interview

- The API Gateway insulates internal microservices from perimeter exposure and protocol churn.
- SSL termination offloads intensive crypto operations from backend compute instances.
- BFF variants dedicate separate gateways tailored specifically to mobile versus web UI needs.

12

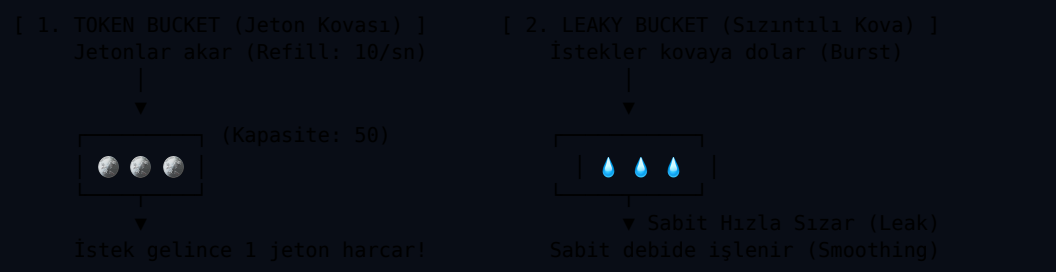
BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

Dağıtık Hız Sınırlama (Rate Limiting): Token Bucket, Leaky Bucket, Sliding Window

Distributed Rate Limiting: Token Bucket, Leaky Bucket & Sliding Window

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir lunapark trenini düşünün: Trene binebilmek için jeton makinesinden jeton almalısınız (Token Bucket). Makine kovaya her saniye 2 yeni jeton atar; kova en fazla 10 jeton alır. Bir anda 10 kişi gelirse hepsi aynı anda trene biner (Burst). Ama 11. kişi kova dolana kadar kapıda beklemek zorundadır! Rate Limiter işte bu lunapark görevlisidir: Kötü niyetli bir robot veya aşırı heyecanlı bir kullanıcı saniyede 10.000 istek atarak sunucuyu çökertmesin diye 'Sakin ol, kotan doldu! (HTTP 429 Too Many Requests)' der!

🇬🇧 ENGLISH (ELI5)

Imagine an amusement park roller coaster ride requiring tokens (Token Bucket). An automated dispenser drops 2 tokens into a bucket every second, up to a max capacity of 10. If a sudden group of 10 arrives, they grab all 10 tokens and board simultaneously (Burst). But person 11 must wait until new tokens drop! The Rate Limiter is that turnstile guard: shielding databases from aggressive bots or buggy loops by replying with HTTP 429 Too Many Requests!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG mülakatlarında en çok sorulan 4 Rate Limiting algoritması: 1) Token Bucket: Amazon ve Stripe'ın tercihidir. Sabit hızda jeton dolar; burst (ani yük) trafiğe izin verir, bellek tüketimi azdır (sadece sayaç + zaman damgası). 2) Leaky Bucket: Trafiği sabit bir çıkış hızında düzleştirir (Traffic Shaping / Smoothing). 3) Fixed Window Counter: Zamanı sabit pencerelere böler (12:00-12:01); pencere sınırında iki katı istek geçmesine izin veren 'Window Boundary' açığı vardır. 4) Sliding Window Log / Counter: Redis ZSET veya bellek sayaçlarıyla kayan pencereyi milisaniye hassasiyetinde tutar; sınır açığını sıfırlar. Dağıtık ortamda Redis + Lua Scripting ile atomik olarak uygulanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The 4 core rate limiting algorithms evaluated in MAANG rounds: 1) Token Bucket: Favored by Amazon/Stripe. Tokens replenish at a fixed rate; effortlessly absorbs short burst traffic with negligible memory (stores only timestamp + token count). 2) Leaky Bucket: Smooths bursty traffic into a constant outflow rate (vital for throttling queues). 3) Fixed Window Counter: Simple counter reset at window boundaries; vulnerable to 2x burst surges across the window boundary. 4) Sliding Window Counter: Blends historical and current window weights to neutralize boundary spikes with minimal Redis memory overhead. Implemented in distributed clusters via atomic Redis Lua scripts.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Thread-Safe Token Bucket Algoritması
import java.time.Instant;

public class TokenBucketRateLimiter {
    private final long capacity;
    private final double refillTokensPerSecond;
    private double availableTokens;
    private long lastRefillTimestamp;

    public TokenBucketRateLimiter(long capacity, double refillTokensPerSecond) {
        this.capacity = capacity;
        this.refillTokensPerSecond = refillTokensPerSecond;
        this.availableTokens = capacity;
        this.lastRefillTimestamp = System.currentTimeMillis();
    }

    public synchronized boolean tryAcquire() {
        refill();
        if (availableTokens >= 1.0) {
            availableTokens -= 1.0;
            return true; // İstek kabul edildi
        }
        return false; // HTTP 429 Too Many Requests
    }

    private void refill() {
        long now = System.currentTimeMillis();
        double elapsedSeconds = (now - lastRefillTimestamp) / 1000.0;
        availableTokens = Math.min(capacity, availableTokens + elapsedSeconds *
refillTokensPerSecond);
        lastRefillTimestamp = now;
    }
}

void main() {
    var limiter = new TokenBucketRateLimiter(3, 1.0); // 3 kapasite, saniyede 1 jeton
    for (int i = 1; i <= 5; i++) {
        boolean allowed = limiter.tryAcquire();
        IO.println("İstek #" + i + " İzin Verildi mi? " + allowed);
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Dağıtık ortamda her web sunucusunun kendi belleğinde local rate limiter tutması mülakatta eksi puandır! 10 sunucu varsa kullanıcı her sunucuya 100 istek atıp toplamda 1.000 istek yapabilir. Hız sınırlayıcı merkezi bir Redis kümesinde veya Envoy filter seviyesinde tutulmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Storing rate limit counters in local JVM memory fails in clustered deployments. 10 pods mean a user can fire 10x their quota via round-robin. Centralize distributed counters within a shared Redis cluster utilizing Lua scripts.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Token Bucket algoritması ani yükleri (bursts) tolere ettiği için endüstri standardıdır.

• Dağıtık ortamlarda yarış durumunu (Race Condition) önlemek için Redis Lua Script kullanılır.

• Kotası dolan istemciye HTTP 429 ve 'Retry-After: 30' yanıt başlığı dönülmelidir.

🎯 Key Takeaways for the Interview

• Token Bucket is the industry standard because it gracefully accommodates bursty traffic spikes.

• Redis Lua scripts execute read-and-decrement steps atomically without distributed locks.

• Rate-limited clients must receive HTTP 429 along with a Retry-After header.